ITI — Information Technology Institute
Software Requirements Specification

Student Examination Management System
Version 1.0 | SQL Server Focus | Database-Centric

Field	Details
Primary Deliverable	SQL Server Database — ERD, Tables, Stored Procedures, Reports
Secondary Deliverable	Optional Desktop App (C++ / Java)
Status	**Final — Ready for Implementation**

1. Introduction & Scope

This SRS defines all requirements for the ITI Student Examination Management System — a database-driven solution for managing exams across ITI branches. All core logic lives in SQL Server stored procedures.

In Scope:

- CRUD for all entities: Branch, Track, Course, Instructor, Student
- Question bank: MCQ (4 options) and True/False
- Random exam generation per course
- Student answer submission and automatic grading
- Three mandatory reports

Out of Scope:

- Web/mobile interface, email notifications, AI features

2. User Roles & Environment

Role	Responsibilities	DB Access
Administrator	Full system management, backups	db_owner
Instructor	Generate exams, manage questions, view reports	Read/Write on exam & question tables
Student	Take exams, view own grades only	db_datareader (read only)

- Database: SQL Server 2019+ via SSMS
- Optional Frontend: C++ (ODBC) or Java (JDBC) — desktop only
- All text columns must use NVARCHAR (Arabic + English support)
- All questions must be pre-loaded before any exam is generated

3. Functional Requirements

3.1 Entity CRUD — REQ-01 to REQ-08 (All CRITICAL)

Stored procedures must exist for Insert, Update, Delete, and Select on every entity:

Entity	Minimum Required Procedures
Branch	InsertBranch, UpdateBranch, DeleteBranch, SelectBranch
Track	InsertTrack, UpdateTrack, DeleteTrack, SelectByBranch
Course	InsertCourse, UpdateCourse, DeleteCourse, SelectByTrack
Instructor	InsertInstructor, UpdateInstructor, AssignInstructorToCourse
Student	InsertStudent, UpdateStudent, DeleteStudent, AssignStudentToTrack
Question	InsertQuestion (MCQ or T/F), UpdateQuestion, DeleteQuestion
Option	InsertOption — 4 per MCQ, 2 per T/F
ModelAnswer	SetModelAnswer — one correct option per question only

3.2 GenerateExam — REQ-09 (CRITICAL)

Property	Specification
Inputs	@CourseID, @ExamName, @NumMCQ, @NumTF
Action	Insert row in Exam + N rows in ExamQuestion with random questions (ORDER BY NEWID())
Rules	No duplicate questions per exam · Assign OrderNo · Raise error if not enough questions

3.3 SubmitExamAnswers — REQ-10 (CRITICAL)

Property	Specification
Inputs	@StudentID, @ExamID, @StartTime, @EndTime, @Answers XML
XML Format	<Answers><Answer><QuestionID>1</QuestionID><ChosenOptionID>5</ChosenOptionID></Answer></Answers>
Action	Insert StudentExam row · Parse XML · Insert StudentAnswer row per answer · Skipped = 0 pts

3.4 CorrectExam — REQ-11 (CRITICAL)

Property	Specification
Input	@StudentExamID
Logic	For each question: if ChosenOptionID = ModelAnswer.OptionID → add Points, else 0
Output	UPDATE StudentExam SET TotalGrade = sum · Wrap in transaction (ROLLBACK on error)

3.5 Reports — REQ-12 to REQ-14 (All CRITICAL)

Procedure	Input	Output Columns
Report_StudentsByDepartment	@DepartmentNo	StudentID, Name, Email, Phone, TrackName, BranchName
Report_StudentGrades	@StudentID	CourseName, ExamName, TotalGrade, MaxDegree, Percentage*
Report_InstructorCourses	@InstructorID	CourseName, TrackName, StudentCount

* Percentage = CAST(TotalGrade AS FLOAT) / MaxDegree * 100 — computed inside SQL.

4. Non-Functional Requirements

ID	Requirement
NFR-01	Exam generation for 50 questions < 2 seconds
NFR-02	Exam correction per student < 1 second
NFR-03	All stored procedures must use transactions — no partial writes
NFR-04	Students cannot access other students' data (enforce via SQL Server roles)
NFR-05	ModelAnswer table must be denied to student role
NFR-06	FK constraints must prevent orphan records — define ON DELETE rules explicitly
NFR-07	All text columns must be NVARCHAR — no VARCHAR on name/text fields
NFR-08	Every SP must have a comment block: purpose, inputs, outputs

5. Database Schema

12 required tables. Every table: PK with IDENTITY(1,1), FK constraints defined, NVARCHAR for all text.

Relationships:

- Branch → Track → Course ↔ Instructor (via Instructor_Course junction)
- Track ↔ Student (via Student_Track junction)
- Course → Exam → ExamQuestion ← Question → Option → ModelAnswer
- Student + Exam → StudentExam → StudentAnswer

Table	Key Columns (name : type)	Notes
Branch	BranchID : INT PK, BranchName : NVARCHAR(100), Location : NVARCHAR(200)	
Track	TrackID : INT PK, TrackName : NVARCHAR(100), BranchID : FK, DurationMonths : INT	
Course	CourseID : INT PK, CourseName : NVARCHAR(100), MinDegree : INT, MaxDegree : INT	
Track_Course	TrackID : FK, CourseID : FK	Junction — composite PK
Instructor	InstructorID : INT PK, InstructorName : NVARCHAR(100), Email : NVARCHAR UNIQUE, DepartmentNo : INT	
Instructor_Course	InstructorID : FK, CourseID : FK	Junction — composite PK
Student	StudentID : INT PK, StudentName : NVARCHAR(100), Email : NVARCHAR UNIQUE, Phone : NVARCHAR(20)	
Student_Track	StudentID : FK, TrackID : FK	Junction — composite PK
Question	QuestionID : INT PK, CourseID : FK, QuestionText : NVARCHAR(MAX), QuestionType : NVARCHAR(10) CHECK IN ('MCQ','TF'), Points : INT	
Option	OptionID : INT PK, QuestionID : FK, OptionText : NVARCHAR(MAX), OptionOrder : INT	4 rows/MCQ, 2 rows/TF
ModelAnswer	ModelAnswerID : INT PK, QuestionID : FK UNIQUE, OptionID : FK	One answer per question
Exam	ExamID : INT PK, ExamName : NVARCHAR(150), CourseID : FK, CreatedDate : DATETIME, TotalQuestions : INT	
ExamQuestion	ExamID : FK, QuestionID : FK, OrderNo : INT	Composite PK
StudentExam	StudentExamID : INT PK, StudentID : FK, ExamID : FK, StartTime : DATETIME, EndTime : DATETIME, TotalGrade : INT	TotalGrade set by CorrectExam
StudentAnswer	StudentAnswerID : INT PK, StudentExamID : FK, QuestionID : FK, ChosenOptionID : FK	

6. Stored Procedures — Full List

Procedure	Inputs	Action	Priority
GenerateExam	@CourseID, @ExamName, @NumMCQ, @NumTF	Create Exam + ExamQuestion (random)	CRITICAL
SubmitExamAnswers	@StudentID, @ExamID, @StartTime, @EndTime, @Answers XML	Create StudentExam + StudentAnswer	CRITICAL
CorrectExam	@StudentExamID	Calculate & save TotalGrade	CRITICAL
Report_StudentsByDepartment	@DepartmentNo	Students filtered by dept	CRITICAL
Report_StudentGrades	@StudentID	Grades + percentage per course	CRITICAL
Report_InstructorCourses	@InstructorID	Courses + student count	CRITICAL
InsertStudent / UpdateStudent / DeleteStudent	Student fields	Basic CRUD	High
InsertQuestion / InsertOption / SetModelAnswer	Question fields	Question bank management	High
AssignInstructorToCourse / AssignStudentToTrack	IDs	Enrollment management	Medium

7. Test Scenarios & Sample Data

#	Test	Expected Result
1	GenerateExam — valid inputs	Exam + ExamQuestion rows created, no duplicates
2	GenerateExam — not enough questions	Error raised, no partial exam created
3	SubmitExamAnswers — all questions answered	StudentExam + all StudentAnswer rows created
4	SubmitExamAnswers — one question skipped	Success — no StudentAnswer for skipped question
5	CorrectExam — all correct	TotalGrade = MaxDegree
6	CorrectExam — all wrong	TotalGrade = 0
7	Run all 3 reports	Correct data returned per report spec
8	Delete Course with existing exams	FK constraint error — delete rejected

Minimum sample data required before testing:

- 3+ Branches · 2+ Tracks each · 5+ Courses · 3+ Instructors assigned to 2+ courses
- 20+ Students across tracks
- 30+ MCQ + 20+ T/F questions — all with options and model answers set

• The project source code must be uploaded and maintained on GitHub for version control and collaboration among team members.

• The project will be hosted on LinkedIn for portfolio and professional visibility purposes.